

IAM - Documentación de Frontend (WEB)

Equipo IAM

22 de octubre de 2025

1 Resumen

Este documento describe los cambios en la verificación de cuenta de la aplicación web (carpeta WEB), incluyendo:

- Llamadas a la API para generar, reenviar y validar códigos.
- Comportamiento estricto en la validación basado en `res.codigo` (B052/B055).
- UX: contador regresivo de 60s y deshabilitado de botones durante el conteo.

2 Servicios

Archivo: WEB/web-app/src/services/usuario.service.ts

```
// Endpoints de verificacin
static generarCodigoVerificacion(usuario: DTO_Usuario): Observable<DTO_Respuesta> {
  return defer(() => api.post<DTO_Respuesta>(API_ENDPOINTS.AUTH.VERIFY.GENERATE, usuario))
    .pipe(map((r) => r.data as DTO_Respuesta));
}
static reenviarCodigoVerificacion(usuario: DTO_Usuario): Observable<DTO_Respuesta> {
  return defer(() => api.post<DTO_Respuesta>(API_ENDPOINTS.AUTH.VERIFY.RESEND, usuario))
    .pipe(map((r) => r.data as DTO_Respuesta));
}
static validarCodigoVerificacion(usuario: DTO_Usuario): Observable<DTO_Respuesta> {
  return defer(() => api.post<DTO_Respuesta>(API_ENDPOINTS.AUTH.VERIFY.VALIDATE, usuario))
    .pipe(map((r) => r.data as DTO_Respuesta));
}
```

3 Pantalla Verify

Archivo: WEB/web-app/src/screens/auth/Verify.tsx. Puntos clave:

- Al entrar, se auto-invoa `generarCodigoVerificacion(user)`.
- En **validar**, no se usa `tipoRespuesta`: se decide por `res.codigo`.
 - Éxito real: B052 (verificado) o B055 (ya verificado).
 - En otros códigos (p.ej. B053/B054), se muestra error y permanece en la pantalla.
- Tras enviar (generar/reenviar), arranca un **countdown de 60s** y se deshabilitan botones mientras corre.

3.1 Validación estricta por res.codigo

```
// Decisin por res.codigo
const cod = ((res as any)?.codigo as string | undefined)?.toUpperCase();
const verificacionOK = cod === "B052" || cod === "B055";

if (!verificacionOK) {
  notificationHelpers.errorAlert(res.mensaje || "Cdigo invlido o vencido");
  return; // No navegar ni cambiar estado/token
}
```

3.2 Actualización de estado y token en éxito

```
// Marcar usuario como activo y guardar token si viene
const updated: DTO_Usuario = {
  ...u,
  estado: { ...u.estado, iD_Estado: 1, ...((u.estado as any)?.ID_Estado !== undefined ? {
    ID_Estado: 1 } : {}) } as any,
};
setUser(updated);
localStorage.setItem("auth_user", JSON.stringify(updated));

const nuevoToken = (res.resultado?.[0] as any)?.accessToken as string | undefined;
if (nuevoToken) localStorage.setItem("accessToken", nuevoToken);
```

3.3 Contador regresivo y deshabilitado de botones

```
// Al enviar cdigo con xito:
startCountdown(60); // deshabilita acciones durante 60s

// Estado que controla UI:
const disabledByCountdown = secondsLeft > 0;
const disableAllActions = loading || disabledByCountdown || !user?.iD_Usuario;
```

4 Mensajería de usuario

- Se muestran toasts de éxito o error según el resultado.
- En los envíos (generar/reenviar), se reconoce `emailEnviado` (agregado por backend) para ajustar el mensaje:
 - `true`: “Código enviado...”.
 - `false`: mensaje informativo (p.ej. “ya verificado”).

5 Pruebas

1. Iniciar sesión, entrar a Verify: debe dispararse “generar”.
2. Revisar correo; en caso de no llegar, revisar Logs de Resend.
3. Probar **código incorrecto**: debe mostrar error y *no* navegar.
4. Probar **código correcto**: debe marcar activo, guardar token y navegar al Home.

6 Buenas prácticas

- La UI *nunca* debe asumir éxito por `tipoRespuesta`: usar `res.codigo`.
- Evitar spam de reenvíos con el *countdown*.
- Mantener `accesToken` solo en flujos de éxito real.